

Managed Airflow / Cloud Composer

Table of Contents

- 1. Introduction 3**
- 2. Airflow and Cloud Composer 3**
- 3. Composer environment and UI 3**
- 4. What is a DAG? 5**
- 5. DAG, Task, Task Instance and DAG Run 5**
- 6. Scheduler, Executor and Worker 5**
- 7. Creating a DAG in Python 6**
- 8. Task dependencies 7**
- 9. DAG scheduling 7**
- 10. Catchup, backfill and retries 8**
- 11. Concurrency and max_active_runs 9**
- 12. Operators and task types 9**
- 13. Your example DAG 10**
- 14. Creating and deploying a DAG in Cloud Composer 10**
- 15. Understanding a DAG Run 11**
- 16. Task states and what to do 12**
- 17. Monitoring a DAG in the Airflow UI 13**
- 18. XCom — passing small values between tasks 13**
- 19. Variables, Connections and Hooks 14**
- 20. Sensors and event waiting 15**
- 21. Branching and trigger rules 15**
- 22. TaskFlow API 15**
- 23. Task Groups, Pools and resource control 17**
- 24. Error handling and retries 17**

25. Debugging a failed task	18
26. Airflow with Dataproc and BigQuery	18
27. Production DAG design	19
28. DAG design principles	20
29. Common beginner mistakes	20
30. End-to-end e-commerce DAG example	21
31. Event-driven scheduling	21
32. Operational checklist	22

1. Introduction

Apache Airflow is a workflow orchestration platform. A DAG defines tasks and their dependencies; the scheduler decides when eligible task instances can run, and workers execute the tasks. Cloud Composer (Managed Service for Apache Airflow) provides a managed Airflow environment on Google Cloud.

Learning approach: Understand the workflow first, then the UI, then DAG code, and finally the production controls. The goal is to make a new engineer capable of creating, reading, running and troubleshooting a DAG.

2. Airflow and Cloud Composer

2.1 Apache Airflow

Airflow is the orchestration layer. It coordinates work performed by Python code and cloud services rather than replacing those processing systems.

2.2 Cloud Composer

Managed Service for Apache Airflow (formerly Cloud Composer) is Google Cloud's managed Airflow service. The supplied example environment is ecom-airflow-env in asia-south1 and shows Airflow 2.11.1. Treat 2.11.1 as the version in the supplied example, not as a universal current default; verify available Airflow versions when creating a new environment.

Cloud Composer ↓ Apache Airflow ↓ DAGs + Scheduler + Workers + Metadata + Airflow UI

Why use Composer? It removes much of the infrastructure-management work so engineers can focus on DAGs, scheduling, integrations and operations.

3. Composer environment and UI

An environment is the managed Airflow home where Airflow components, workers, DAG files and supporting services operate.

Figure 1 — Managed Airflow / Cloud Composer Environments page from the supplied project.

3.1 Opening the Airflow UI

Open Google Cloud Console and select the correct project.

Open Managed Airflow / Cloud Composer → Environments.

Open the required environment.

Use the Airflow web UI link.

Open DAGs to view the available workflows.

Scenario: With separate development, test and production environments, always verify the environment before changing DAG state or triggering a production run.

4. What is a DAG?

DAG means Directed Acyclic Graph. Practically, it is a workflow made of tasks connected by dependencies. The dependency graph must not contain a cycle.

extract → transform → load

The important part is the dependency graph. The order in which tasks are written in a Python file is not, by itself, the execution order.

5. DAG, Task, Task Instance and DAG Run

Mental model: DAG = blueprint; DAG Run = one execution; Task = one step; Task Instance = that step in one particular run.

6. Scheduler, Executor and Worker

Airflow separates scheduling from task execution. The scheduler monitors DAGs and determines which tasks are ready. The executor controls how tasks are submitted, while workers provide execution capacity.

Why this matters: A task that is not running may be blocked by dependencies, scheduling, worker capacity, pools, external systems or configuration—not necessarily by its Python code.

6.1 How scheduling and execution work together

A common beginner assumption is that the scheduler itself runs the Python code. A better mental model is that the scheduler decides which DAG runs and task instances are eligible, while the execution mechanism sends eligible work to the place that actually executes the task.

DAG parsed → Scheduler evaluates schedule/dependencies → task eligible → execution mechanism submits work → worker executes task

6.2 Why a task can remain waiting

An upstream dependency has not completed successfully.

The DAG has reached a concurrency limit such as `max_active_runs`.

A pool has no available slot.

Execution capacity is temporarily unavailable.

The task is waiting for an external condition.

Workflow logic intentionally prevented the task from running.

Scenario: A task appears ready but stays queued. Check dependencies, pools and execution capacity before changing the Python code.

7. Creating a DAG in Python

A DAG is defined in a Python file. At minimum, define the DAG, create tasks and establish dependencies.

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

with DAG( dag_id="sales_pipeline_dag", start_date=datetime(2026, 1, 1), schedule="@daily", catchup=False ) as dag:
    extract = PythonOperator( task_id="extract_data", python_callable=extract_data )
    transform = PythonOperator( task_id="transform_data", python_callable=transform_data )
    load = PythonOperator( task_id="load_data", python_callable=load_data )
    extract >> transform >> load
```

8. Task dependencies

Dependencies tell Airflow when a task becomes eligible to run.

```
extract >> transform >> load
```

```
extract >> [validate, audit] validate >> load audit >> load
```

In the second graph, validate and audit can run after extract, while load waits for both. This relationship is what controls execution order.

Common mistake: Do not infer dependency order from the order in which tasks appear in the Python file. Check the dependency expressions or the Graph view.

9. DAG scheduling

A schedule defines when Airflow creates/schedules runs. Cron expressions and presets such as @daily are common choices.

```
schedule="0 0 * * *" schedule="@daily"
```

9.1 Start date and logical dates

Airflow schedules runs based on the DAG timetable and start date. A scheduled run represents a logical date/data interval; for interval-based schedules, execution occurs after the interval is complete.

Scenario: A daily run for an August 31 data interval may execute after the August 31 interval closes. The logical date identifies the interval being processed, not simply the wall-clock start time.

9.2 Reading cron expressions

Cron schedules use five fields: minute, hour, day of month, month and day of week. The schedule must also be interpreted in the DAG timezone.

```
0 0 * * * → every day at 00:00 0 */2 * * * → every 2 hours 30 6 * * 1-5 → 06:30 Monday-Friday
```

Practical warning: A cron expression can be syntactically correct but still be wrong for the business if the timezone or intended data interval is misunderstood.

10. Catchup, backfill and retries

10.1 Catchup

Catchup controls whether missed scheduled intervals are created from the DAG's start date through the current period. With catchup=False, old intervals are not automatically created.

10.2 Backfill

Backfill is an intentional way to create historical runs for a selected date range, useful when historical data needs to be reprocessed.

10.3 Retries

Retries give a failed task additional attempts. They are useful for transient failures but should not be used to hide deterministic code, SQL, IAM or data-quality problems.

```
retries=2 retry_delay=timedelta(minutes=5)
```

Use retries for: Transient network/service failures or other recoverable conditions. Read the logs before increasing retries.

10.4 Catchup vs backfill

Catchup is automatic scheduling behavior controlled by the DAG configuration. Backfill is an intentional historical-processing action. Catchup asks whether missed intervals should be created automatically; backfill means an engineer explicitly requests historical runs.

10.5 Designing safe retries

A retry repeats task execution. If a task has a non-idempotent side effect, the retry can repeat the business operation. For example, blindly inserting the same order twice can create duplicates. Prefer deterministic writes, stable business keys, or MERGE/UPSERT-style patterns where appropriate.

Unsafe retry: INSERT a new business row every attempt Safer pattern: MERGE/UPSERT using a stable business key

11. Concurrency and max_active_runs

max_active_runs limits how many DAG runs can be active simultaneously. The supplied example shows max_active_runs = 1.

max_active_runs=1

Scenario: If a daily financial load takes 45 minutes, limiting active runs can prevent overlapping runs from creating conflicting or duplicate business work.

11.1 max_active_runs vs other controls

max_active_runs limits how many active DAG runs can exist for the same DAG. It does not mean only one task can execute. Pools, task-level limits and environment execution capacity solve different concurrency problems.

12. Operators and task types

Operators are reusable task definitions. Choose one based on the system or processing action the task needs to perform.

Rule: Airflow orchestrates; the operator provides the interface to the actual work.

12.1 From operator definition to execution

An operator instance in a DAG creates a task definition. When a DAG Run is created, Airflow creates a task instance for that run. The instance receives a state and is eventually executed.

Operator object → Task definition → DAG Run → Task Instance → Execution → State/logs

Example: BigQueryInsertJobOperator defines the orchestration task; BigQuery performs the actual SQL job; Airflow tracks the task state.

13. Your example DAG

Figure 2 — Airflow DAG details page for ecom_continuous_lakehouse_dag from the supplied environment.

The supplied screenshot shows ecom_continuous_lakehouse_dag with four displayed runs, all successful, three tasks, schedule 0,12 * * *, max_active_runs = 1, catchup = false and UTC displayed.

Important: The screenshot shows task names but not dependency arrows. Use Graph view or the Python DAG code for the exact execution order.

14. Creating and deploying a DAG in Cloud Composer

A Composer DAG deployment has two main parts: create a valid DAG Python file and place it in the environment's DAGs folder.

14.1 Prepare the DAG file

Create a Python file such as sales_pipeline_dag.py.

Define the DAG, tasks and dependencies.

Check imports and syntax before deployment.

Test task logic and SQL separately where practical.

14.2 Upload the DAG

Open the required Cloud Composer environment.

Open the environment's DAGs folder.

Upload the Python DAG file.

Wait for Airflow to parse the file.

Open the Airflow UI and locate the DAG.

14.3 Enable and trigger

Confirm the DAG is not unintentionally paused.

Wait for its schedule or use Trigger DAG for a manual run.

Open the run and inspect Graph, task states and logs.

Production caution: Do not manually trigger a production DAG without understanding its date parameters, inputs, idempotency and downstream side effects.

14.4 What happens after uploading a DAG?

Composer makes the DAG file available to the Airflow environment. Airflow parses the file and tries to discover valid DAG objects. A syntax error, missing import, invalid definition or incompatible provider dependency can prevent the DAG from appearing correctly.

14.5 Import error vs task failure

15. Understanding a DAG Run

A DAG Run is one execution of the DAG. It contains task instances, each with a state, duration and log output.

DAG ■■■ DAG Run: 2026-08-26 12:00 UTC ■■■ task_a → success ■■■ task_b → success ■■■ task_c → success

15.1 Scheduled run vs manual run

A scheduled run comes from the DAG timetable. A manual run is started intentionally through the UI or API. Manual runs can happen outside the normal schedule and therefore deserve special attention to run configuration, logical date and downstream side effects.

Scenario: A daily load processes the previous day's partition. A manual run without understanding date handling can reload the wrong interval or duplicate business data.

16. Task states and what to do

16.1 Typical task lifecycle

scheduled → queued → running → success ■ failed → up_for_retry → queued/running ■ skipped

Use the current state as a diagnostic clue. 'Queued' points toward execution capacity or pool issues; 'upstream_failed' points toward a dependency; 'failed' points toward the task's own log and any delegated service.

17. Monitoring a DAG in the Airflow UI

The DAG page contains views that answer different operational questions.

For a new user: Use Graph to understand workflow order and Logs to understand why a task failed.

17.1 Practical monitoring workflow

Start with the DAG Run state.

Open Graph view and locate the task or branch blocking progress.

Inspect the task instance state, duration and try number.

Read the task log and find the first meaningful error.

Check Dataproc, BigQuery or other external logs when the task delegates work.

After recovery, validate the business output.

Important: A green task confirms technical execution for that task; it does not automatically prove the business data is correct.

18. XCom — passing small values between tasks

XCom (cross-communication) lets tasks share small pieces of metadata. It is useful when one task produces a small value another task needs.

```
# Push context["ti"].xcom_push( key="file_name", value="orders_2026_08_26.csv" ) # Pull file_name =  
context["ti"].xcom_pull( task_ids="extract_file", key="file_name" )
```

Do not use XCom for large datasets. Store large data in Cloud Storage, BigQuery or another appropriate data store and pass only identifiers/metadata through XCom.

18.1 What belongs in XCom

Good XCom values are small metadata such as filenames, object paths, IDs, flags or small counts. Large data should stay in Cloud Storage, BigQuery or another data system. Pass a reference rather than the data itself.

18.2 Why large XCom values are a bad design

XCom values are stored as Airflow metadata and are intended for communication between tasks, not for moving datasets. Large payloads increase metadata storage and operational complexity.

19. Variables, Connections and Hooks

19.1 Variables

Variables store configurable values such as environment flags, bucket names or other non-secret configuration.

19.2 Connections

Connections store configuration and credentials used by operators/hooks to access external systems.

19.3 Hooks

Hooks provide reusable interfaces to external systems and are commonly used inside operators.

Security practice: Do not hard-code secrets into DAG files. Use connections and the environment's supported secret-management approach.

19.4 Choosing the right feature

Scenario: The bucket name can come from configuration, the approved connection/identity provides access, and a provider Hook handles the service interaction.

20. Sensors and event waiting

Sensors wait for a condition such as a file arrival, external task completion or external resource availability.

Why use sensors? They express 'wait until this condition occurs' directly in the workflow.

Design warning: Long-running sensors can consume execution capacity if used inefficiently. Use deferrable sensors/operators when supported and appropriate.

20.1 How a sensor behaves

A sensor checks a condition until it succeeds, fails or reaches a timeout. Traditional sensors can occupy execution capacity while waiting. Deferrable sensors move long waits into a deferred state when the provider supports that behavior.

check condition → not ready → wait/defer → check again → ready → continue

Design choice: Use a sensor when the workflow must observe an external condition. If the source can emit a reliable readiness event, event-driven triggering may be cleaner than repeated polling.

21. Branching and trigger rules

21.1 Branching

Branching allows a DAG to choose different downstream paths based on a condition.

→ full_load check_file → incremental_load

21.2 Trigger rules

By default, downstream tasks wait for upstream success. Trigger rules let you express alternate conditions, such as running cleanup when upstream tasks fail or are skipped.

Scenario: A cleanup task may need to run even if the processing task failed. A suitable trigger rule can model that requirement.

21.3 Why trigger rules matter

A downstream task normally waits for successful upstream tasks. Trigger rules allow different conditions. A cleanup task, for example, may need to run even when an upstream processing task fails.

process → success → publish ■ failed → cleanup

22. TaskFlow API

The TaskFlow API provides a function-oriented way to define Python tasks and automatically pass small outputs between tasks using XCom.

```
from airflow.decorators import dag, task
from datetime import datetime

@dag(start_date=datetime(2026, 1, 1),
     schedule="@daily",
     catchup=False)
def sales_pipeline():
    @task
    def extract():
        return "orders.csv"
    @task
    def transform(file_name):
        return file_name.replace(".csv", "_clean.csv")
    @task
    def load(file_name):
        print(file_name)
    load(transform(extract()))
sales_pipeline()
```

When to use: TaskFlow is useful for Python-centric workflows; traditional provider operators remain valuable for BigQuery, Dataproc and other managed-service actions.

22.1 When TaskFlow is useful

TaskFlow is particularly useful for Python-centric workflows because normal function calls can express dependencies. Specialized provider operators remain useful when the task is fundamentally a call to BigQuery, Dataproc or another managed service.

Trade-off: TaskFlow reduces Python boilerplate; it does not replace provider operators.

23. Task Groups, Pools and resource control

23.1 Task Groups

Task Groups visually organize related tasks in the UI and make larger DAGs easier to navigate.

24.2 Pools

Pools limit how many tasks can use a constrained resource at the same time.

Scenario: If an external database safely supports only five concurrent extraction jobs, a pool can prevent the DAG fleet from overwhelming it.

24.3 Pools as a safety mechanism

Pools are a guardrail for constrained shared resources. Instead of letting every scheduled run call an external system simultaneously, a pool limits how many tasks can use that resource at the same time.

Scenario: A vendor API safely supports only 10 concurrent requests. Assign the API tasks to a 10-slot pool to prevent the workflow fleet from overwhelming the service.

24. Error handling and retries

Rule: Always read the task log before Retry. A retry can recover a transient failure, but it cannot fix deterministic errors.

25.1 Retry, delay and timeout are different

Design point: Retry policies should match the failure pattern. Fast retries during an outage can increase pressure; long delays can postpone recovery.

25. Debugging a failed task

Open the DAG run.

Identify the failed task.

Open the task log and find the first meaningful error.

Classify the issue: code, data, IAM, dependency, worker/resource or external service.

Check related Dataproc/BigQuery/service logs if applicable.

Fix the root cause.

Retry or rerun according to the recovery process.

Validate the business output.

Task log → first meaningful error → root cause → fix → retry/rerun → validate

26.1 Find the system that actually failed

The Airflow task can be the messenger rather than the root cause. If the task submits a Dataproc job and that job fails, the detailed failure may be in Dataproc logs. Apply the same logic to BigQuery and other external services.

Airflow task log → identify delegated system → inspect system logs → fix root cause → retry/rerun

26. Airflow with Dataproc and BigQuery

The supplied project uses PythonOperator, DataprocCreateBatchOperator and BigQueryInsertJobOperator. These components illustrate the separation between orchestration and processing.

Airflow = Orchestration Dataproc = Spark processing BigQuery = SQL analytics GCS = Storage

27.1 Example execution sequence

Scheduler ↓ Dataproc task submits Spark/PySpark job ↓ Dataproc completes successfully ↓ BigQuery task runs ↓ Validation / notification

The DAG should model the dependency on the external job completing successfully. Airflow coordinates; Dataproc and BigQuery perform specialized processing.

27. Production DAG design

A production DAG needs more than correct Python code.

Business purpose and expected output.

Source and destination systems.

Schedule, timezone and data interval semantics.

Task dependencies.

Retries, timeouts and failure handling.

Idempotency and duplicate-processing strategy.

IAM/service accounts and connections.

Variables and environment-specific configuration.

Pools/concurrency limits where needed.

Monitoring, alerting and operational ownership.

Backfill/replay strategy.

Deployment and version-control process.

28.1 Idempotency in practical terms

Idempotency means a retry or rerun does not create an unintended second business effect. This is critical because Airflow supports retries, manual reruns and historical processing.

28.2 Why timeouts and alerts matter

Production workflows should not wait forever. Sensible task/sensor timeouts convert silent hangs into visible failures, while alerting tells the owner that action is required.

28. DAG design principles

Keep heavy processing in the appropriate processing service.

Make tasks observable and independently retryable where practical.

Prefer idempotent operations.

Avoid hard-coded credentials.

Do not use XCom for large payloads.

Use clear IDs and naming conventions.

Avoid unnecessary high-frequency polling.

Document branching, trigger rules and unusual dependencies.

29.1 Keep orchestration separate from processing

Airflow should normally coordinate work rather than become the data-processing engine for large workloads. Delegate heavy computation to the service designed for it and let Airflow track the dependency and outcome.

Why? This separation makes workflows easier to scale, observe, retry and troubleshoot.

29. Common beginner mistakes

Thinking Airflow, Dataproc and BigQuery are the same thing.

Confusing DAG, DAG Run and Task Instance.

Assuming Python file order defines execution order.

Misreading UTC as local time.

Ignoring logical date/data-interval semantics.

Enabling catchup without understanding historical runs.

Retrying failures without reading logs.

Putting large data in XCom.

Hard-coding secrets.

Allowing duplicate business processing without an idempotency strategy.

Building one giant task instead of observable workflow steps.

30.1 Mistakes that create operational problems

The most common operational mistake is treating every failure the same. A code error, transient API failure, bad input file, IAM error and external job failure require different actions.

Read the log before retrying.

Check the actual dependency graph.

Verify the logical date before a manual rerun.

Check external service logs when Airflow delegates the work.

Validate data after recovery.

30. End-to-end e-commerce DAG example

DAG Run ■ ▼ Extract / Python task ■ ▼ Dataproc / PySpark ■ ▼ BigQuery SQL / Load ■ ▼ Success / Alert

Airflow decides when the workflow runs and what depends on what. Dataproc and BigQuery perform specialized processing.

31.1 Scenario

New sales data is expected twice per day. The DAG triggers at the scheduled times, submits PySpark work to Dataproc, and starts the BigQuery step only after Dataproc succeeds. If Dataproc fails, the downstream task should remain blocked unless the DAG explicitly defines another recovery path.

31.2 Detailed e-commerce flow

A scheduled run can start a Python extraction task, then submit a Dataproc processing task, and finally run a BigQuery step. Each downstream task should wait for the upstream business condition it depends on.

Schedule → Extract → Dataproc → BigQuery → Validate/Notify

Scenario: If Dataproc fails, BigQuery should not start with incomplete data. A dependency such as `dataproc_task >> bigquery_task` expresses that rule.

31. Event-driven scheduling

Modern Airflow deployments can also react to external events rather than relying only on fixed schedules. Event-driven scheduling is useful when an actual change or event should be the business trigger.

Decision rule: Use time-based scheduling when the business process is time-based. Use event-driven scheduling when the arrival/change event is the real trigger.

32.1 Schedule vs sensor vs event trigger

Design principle: Do not poll every minute just because polling is familiar. When the source can reliably signal readiness, an event-driven workflow can remove unnecessary waiting.

32. Operational checklist

Reference Tables

UI item	Meaning	Why it matters
Environment name	Name of the managed Airflow environment.	Identifies the environment.
Location	Google Cloud region.	Determines where the environment is deployed.
Airflow version	Airflow version used by the environment.	Important for compatibility.
Airflow	Opens the Airflow web UI.	Main workflow operations interface.
DAGs	Shows workflows.	Use it to inspect and operate DAGs.
Logs	Environment logs.	Useful for infrastructure troubleshooting.

Term	Meaning	Example
DAG	Complete workflow definition.	E-commerce pipeline.
Task	One unit of work.	Clean data.
DAG Run	One execution of the DAG.	12:00 UTC run.
Task Instance	A task during one specific DAG Run.	Cleaning task in the 12:00 run.
Operator	Class used to define a task's behavior.	PythonOperator.

Component	Main responsibility
Scheduler	Creates/schedules DAG runs and schedules ready task instances.
Executor	Controls how tasks are executed.
Worker	Executes task work.
Metadata database	Stores workflow state and metadata.
Airflow UI / API	Provides operational access to DAGs, runs and tasks.

Setting	Purpose
dag_id	Unique DAG identifier.
start_date	Defines the starting point for scheduling/data intervals.
schedule	Defines the timetable.
catchup	Controls creation of missed historical scheduled runs.
task_id	Unique task identifier inside the DAG.
python_callable	Function executed by PythonOperator.
>>	Defines a dependency from upstream to downstream.

Control	What it controls	Example
max_active_runs	Active runs of one DAG.	Only one daily load active at a time.
Pool slots	Tasks sharing a constrained resource.	Only five API/database calls at once.
Execution capacity	How much task work can run simultaneously.	Excess tasks remain queued.

Operator	Typical use
PythonOperator	Run a Python callable.

Operator	Typical use
BigQuery operators	Submit/manage BigQuery jobs.
Dataproc operators	Submit/manage Spark or PySpark processing.
BashOperator	Run shell commands.
Sensors	Wait for a condition or external event.
TaskFlow API	Define Python tasks as functions with automatic small-value handoffs.

Observed item	Meaning
DAG ID	ecom_continuous_lakehouse_dag
Schedule	0,12 * * * — twice per day.
Displayed runs	4
Displayed result	4 successful
Tasks	3
Timezone	UTC
max_active_runs	1
catchup	false

Problem	Meaning	Where to investigate
Import/parse error	Airflow could not load the DAG definition.	DAG import errors / environment logs.
Task failure	The DAG loaded but a task failed during execution.	Task instance log.
External job failure	A delegated Dataproc/BigQuery job failed.	Airflow log + external service logs.

Run information	Why it matters
Run ID	Identifies the particular execution.
Logical date	Identifies the run's logical data interval.
Start/end time	Shows actual execution timing.
Duration	Helps identify unusual runtime changes.
Task states	Shows which steps succeeded or failed.
Event log	Shows task/DAG state transitions and events.

State	Meaning	First action
scheduled	Ready to be considered for execution.	Check dependencies/scheduler if it does not progress.
queued	Waiting for execution capacity.	Check workers/executor/pools.
running	Executing now.	Inspect logs if it runs unusually long.
success	Finished successfully.	Validate the output when needed.
failed	Execution failed.	Read the first meaningful log error.
upstream_failed	A previous dependency failed.	Fix upstream failure first.
up_for_retry	A retry is scheduled.	Determine whether failure is transient.
skipped	Task was intentionally skipped.	Inspect branching/trigger rules if unexpected.

View	Use it for
Details	DAG metadata, recent runs and summary.
Graph	Understand task dependencies and workflow structure.
Gantt	See task timing and where execution time is spent.
Code	Inspect the parsed DAG source.
Event Log	Review state and operational events.
Run Duration	Compare run durations.
Task Duration	Identify task runtime changes.
Calendar	Review runs over dates.

Feature	Use it for	Avoid using it for
Variable	Non-secret configuration.	Passwords/tokens or datasets.
Connection	External-system connection details and credentials.	Business data.
Hook	Reusable interface to an external service.	DAG scheduling logic.

Failure type	Typical response
Transient network/service issue	Retry with a sensible delay.
Invalid input/data	Fix, quarantine or reject the data according to the design.
SQL/code error	Correct the query/code.
IAM/permission failure	Fix the service account/role configuration.
External dependency unavailable	Use retry/sensor strategy and monitor the dependency.

Setting	Purpose
retries	Number of additional attempts.
retry_delay	Wait before another attempt.
execution timeout	Maximum allowed task runtime.
sensor timeout	Maximum time a sensor may wait.

Service	Role
Cloud Composer / Airflow	Orchestrates and tracks the workflow.
Dataproc	Runs distributed Spark/PySpark processing.
BigQuery	Runs analytical SQL/warehouse processing.
Cloud Storage	Stores files and objects.

Work type	Safer design
File ingestion	Track processed files or use a deterministic target.
Warehouse load	Use deterministic partitions and MERGE/replace logic where appropriate.
API side effect	Use an idempotency key when supported.
Database update	Use stable business keys and update/merge logic.

Requirement	Starting point
Run at a fixed time	Time-based schedule.
Run whenever an external event occurs	Event-driven trigger.
Wait for a condition that has no event mechanism	Sensor/deferrable wait.
Reprocess a historical interval	Backfill.

Check	Verify
Environment	Correct project and Composer environment.
DAG state	Enabled/not unintentionally paused.
Schedule	Correct cron/preset and timezone.
Dependencies	Graph matches expected order.
Run state	No unexpected queued/failed/retrying tasks.
Logs	Root cause identified for failures.
External jobs	Dataproc/BigQuery work completed as expected.
Data quality	Output is correct, not merely technically successful.
Concurrency	No unsafe overlapping runs.
Recovery	Retry/backfill/replay strategy understood.